



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

Capítulo 7

Modelagem do Banco de Dados para o Sistema de Delivery

Resumo

Nesta unidade avançamos na construção da aplicação *delivery* incorporando a camada de persistência de dados por meio do **Flask-SQLAlchemy** e do modelo relacional. Partindo da modelagem conceitual do sistema, construiremos entidades, relacionamentos e restrições utilizando ORM (*Object-Relational Mapping*), integrando a representação orientada a objetos em Python com a estrutura relacional do banco de dados. Configuramos a infraestrutura de banco de dados da aplicação, organizamos os modelos segundo uma arquitetura modular e implementamos entidades centrais como usuários, papéis, associações de permissões e estruturas de localização. Também discutiremos conceitos fundamentais como chaves primárias, chaves estrangeiras, relacionamentos muitos-para-muitos, integridade referencial, cascatas de remoção e modelagem temporal de associações.

*“Modelar dados não significa apenas armazenar informações, mas
representar corretamente as relações e regras que
governam o domínio da aplicação.”*

Adaptado da filosofia de modelagem relacional de Edgar F. Codd

Introdução

Nesta unidade, avançamos da modelagem conceitual para a definição estruturada do banco de dados do sistema de delivery, adotando como base o modelo relacional.

Bancos de dados relacionais organizam informações por meio de tabelas, cada uma representando uma entidade do domínio da aplicação. No contexto de um sistema de delivery, surgem naturalmente entidades como usuários, restaurantes, produtos e pedidos. Cada uma dessas entidades passa a ser descrita por uma tabela específica, cuja estrutura é fixa, enquanto seus dados evoluem ao longo do tempo.

De forma mais precisa, uma tabela pode ser interpretada como uma relação

$$R(A_1, A_2, \dots, A_n),$$

em que cada A_i representa um atributo da entidade modelada. Cada linha da tabela corresponde a uma tupla

$$t = (a_1, a_2, \dots, a_n),$$

onde cada valor a_i pertence ao domínio do atributo A_i . Essa formalização nos permite tratar o banco de dados não apenas como um repositório de dados, mas como uma estrutura matemática com propriedades bem definidas.

Um elemento central dessa estrutura é a noção de chave primária. Toda relação possui um subconjunto de atributos capaz de identificar unicamente cada tupla. Formalmente, uma chave primária $K \subseteq \{A_1, \dots, A_n\}$ deve satisfazer duas propriedades fundamentais: unicidade, isto é, nenhuma duas tuplas compartilham o mesmo valor em K , e minimalidade, no sentido de que nenhum subconjunto próprio de K preserva essa propriedade. Na prática, essa abstração é frequentemente materializada pelo atributo `id`, amplamente utilizado nas tabelas do sistema.

Além da identificação interna das tuplas, o modelo relacional permite estabelecer vínculos entre diferentes relações. Isso é feito por meio de chaves estrangeiras, que conectam atributos de uma tabela à chave primária de outra. Se uma relação R possui um atributo A_i que referencia a chave primária K de uma relação S , representamos esse vínculo por

$$A_i \rightarrow S(K).$$

Esse mecanismo define, de forma precisa, os relacionamentos entre entidades e garante a chamada integridade referencial do banco de dados.

No contexto do sistema de delivery, tais relacionamentos aparecem de maneira natural: pedidos estão associados a usuários, produtos estão associados a restaurantes, e entregas estão associadas a pedidos. Essa rede de dependências pode ser interpretada como um grafo direcionado, no qual as tabelas desempenham o papel de vértices, enquanto as chaves estrangeiras correspondem às arestas que conectam essas estruturas. As restrições impostas pelo modelo garantem que esse grafo permaneça consistente ao longo da evolução do sistema.

Direcionamento da Unidade

Com base nessa estrutura conceitual, construiremos o modelo de dados do sistema de forma progressiva, buscando equilibrar rigor estrutural e flexibilidade de evolução. Partiremos de uma entidade central de identidade, o usuário, responsável por representar qualquer ator do sistema. A partir dela, introduziremos o conceito de papéis, permitindo que um mesmo usuário assuma diferentes funções ao longo do tempo, como cliente, entregador ou administrador.

Essa separação será modelada por meio de uma estrutura composta pelas entidades **User**, **Role** e pela relação de associação **RoleUser**, que permitirá representar vínculos do tipo muitos-para-muitos, inclusive com extensão temporal. Em paralelo, incorporaremos uma estrutura de localização baseada nas entidades **Address** e **City**, garantindo consistência e reutilização de informações geográficas.

A partir dessa base, serão introduzidas as entidades específicas do domínio de delivery, sempre respeitando princípios de integridade referencial e coerência semântica. O objetivo não é apenas construir um banco de dados funcional, mas desenvolver uma compreensão estrutural que permita sua evolução controlada ao longo do tempo.

Toda entidade operacional do sistema deve, sempre que possível, ser derivada de um usuário.

Essa diretriz implica que elementos como restaurante, cliente e entregador não devem ser

tratados como entidades independentes, mas sim como especializações de uma entidade central de identidade, o que favorece consistência, extensibilidade e simplificação do modelo.

Frameworks de Banco de Dados em Python

A implementação de um banco de dados em uma aplicação Python envolve duas decisões fundamentais: a escolha do sistema gerenciador de banco de dados (SGBD) e a definição da camada responsável pelo acesso aos dados.

O ecossistema Python oferece suporte a uma ampla variedade de bancos de dados, incluindo sistemas relacionais, como MySQL, PostgreSQL e SQLite, e soluções não relacionais, como MongoDB, CouchDB e Redis. No contexto do Flask, essa escolha é amplamente flexível, uma vez que o framework não impõe restrições quanto à tecnologia adotada.

No entanto, o ponto central da arquitetura não está apenas no banco escolhido, mas na forma como a aplicação interage com ele. Nesse sentido, é comum introduzir uma camada de abstração que permita manipular dados por meio de objetos Python, em vez de comandos SQL explícitos. Essas camadas são conhecidas como ORMs (Object-Relational Mappers), no caso de bancos relacionais, e ODMs (Object-Document Mappers), para bancos orientados a documentos.

No caso de ORMs, cada classe Python representa uma tabela do banco de dados, enquanto cada instância corresponde a uma tupla da relação. Essa correspondência pode ser formalizada como uma aplicação

$$\mathcal{M} : \text{Classes} \rightarrow \text{Relações},$$

na qual operações sobre objetos são automaticamente traduzidas em operações no banco de dados. Com isso, o desenvolvedor passa a trabalhar em um nível mais alto de abstração, alinhado ao paradigma orientado a objetos.

A escolha de um framework de banco de dados, portanto, deve ser guiada por alguns critérios fundamentais que envolvem não apenas aspectos técnicos, mas também implicações

no processo de desenvolvimento.

Um primeiro aspecto é a facilidade de uso. Camadas de abstração como ORMs oferecem uma interface mais expressiva e integrada ao código Python, eliminando a necessidade de escrever consultas SQL manualmente na maioria dos casos. Essa abordagem reduz a probabilidade de erros e favorece a organização do código, especialmente em sistemas complexos, nos quais múltiplas entidades e relacionamentos precisam ser manipulados de forma consistente.

Por outro lado, essa abstração não é gratuita. A tradução entre o domínio orientado a objetos e o domínio relacional introduz uma sobrecarga computacional. Em geral, essa penalidade de desempenho é pequena e, na maioria dos cenários, desprezível quando comparada ao ganho de produtividade. Ainda assim, é importante que o framework escolhido permita acesso direto ao banco de dados sempre que necessário, possibilitando a otimização de operações críticas por meio de consultas nativas. Dessa forma, evita-se um falso dilema entre produtividade e desempenho.

Outro fator relevante é a portabilidade. Essa questão se manifesta em dois níveis. Em primeiro lugar, é necessário considerar se o banco de dados escolhido é compatível com os ambientes de desenvolvimento e produção, especialmente em cenários de implantação em nuvem. Em segundo lugar, deve-se avaliar o grau de independência oferecido pelo framework em relação ao SGBD. Algumas ferramentas são específicas de um único banco, enquanto outras — como o SQLAlchemy — oferecem uma camada de abstração capaz de operar sobre diferentes sistemas relacionais, como MySQL, PostgreSQL e SQLite, com mudanças mínimas no código da aplicação.

Por fim, embora não seja estritamente necessário, a integração com o Flask constitui um fator prático importante. Frameworks desenvolvidos como extensões do Flask simplificam aspectos como configuração, gerenciamento de conexões e ciclo de vida das sessões, reduzindo a necessidade de código adicional e favorecendo uma arquitetura mais limpa.

Escolha Arquitetural

Considerando esses critérios, adotamos neste projeto o uso do **Flask-SQLAlchemy**, uma extensão do Flask que encapsula o SQLAlchemy e fornece uma interface integrada para o mapeamento objeto-relacional.

Essa escolha permite representar diretamente, como classes Python, as entidades do sistema de delivery — como usuários, restaurantes, produtos e pedidos — preservando simultaneamente a estrutura do modelo relacional e a expressividade do paradigma orientado a objetos. Mais do que uma conveniência de implementação, essa correspondência estabelece uma ponte conceitual entre o modelo de dados e o código da aplicação.

Além disso, o SQLAlchemy oferece suporte a múltiplos bancos relacionais e mantém a possibilidade de execução de consultas SQL explícitas quando necessário, garantindo flexibilidade para evolução e otimização do sistema.

Essa base será explorada nas próximas seções, nas quais definiremos as entidades do sistema diretamente no ORM, consolidando a transição entre modelagem conceitual e implementação.

Gerenciamento de Banco de Dados com Flask-SQLAlchemy

A integração entre a aplicação Flask e o banco de dados será realizada por meio da extensão **Flask-SQLAlchemy**. Essa extensão atua como uma camada intermediária que simplifica o uso do SQLAlchemy em aplicações Flask, oferecendo tanto um mapeamento objeto-relacional de alto nível quanto acesso direto a comandos SQL quando necessário.

Diferentemente de exemplos simplificados frequentemente apresentados na literatura, adotaremos desde o início uma organização compatível com aplicações reais. Em particular, o banco de dados não será inicializado diretamente no arquivo principal da aplicação, mas sim como uma extensão desacoplada, integrada posteriormente ao objeto Flask.

Instalação

A biblioteca pode ser instalada via pip no ambiente virtual do projeto:

```
(venv) $ pip install flask-sqlalchemy
```

Verifique ainda se o arquivo `pyproject.toml` foi atualizado na sua posição principal, ou seja:

Código: Trecho do `pyproject.toml`

```
1 [project]
2 name = "delivery"
3 # major (mudanca grande), minor (versao trabalhada), patch (correcao bug)
4 version = "0.1.0"
5 description = "A web platform for managing food delivery orders and restaurant
6               operations."
7 readme = "README.md"
8 requires-python = ">=3.9"
9 license = {file = "LICENSE"}
10 authors = [
11     {name = "Wanderson Santana", email = "wanderson.santana@uvv.br"}
12 ]
13 dependencies = [
14     "Flask",
15     "dotenv",
16     "flask-wtf",
17     "email-validator",
18     "flask-sqlalchemy", # Inclusao da nova biblioteca aqui
19 ]
```

Configuração do Banco de Dados

No SQLAlchemy, a conexão com o banco de dados é definida por meio de uma URL, que codifica o tipo de banco utilizado, credenciais de acesso e a localização do serviço. De forma geral, essa URL segue o padrão:


```
1 dialect://username:password@host/database
```

Em bancos relacionais tradicionais, como MySQL e PostgreSQL, todos esses elementos estão presentes. Já no caso do SQLite, por se tratar de um banco embarcado, a URL reduz-se essencialmente ao caminho de um arquivo no sistema de arquivos.

Nesta unidade, adotaremos inicialmente o SQLite, pois sua configuração é simples e não requer um servidor dedicado. Essa escolha permite concentrar o foco na modelagem de dados e na integração com a aplicação Flask.

Diferentemente de abordagens mais simplificadas, a configuração do banco de dados não será definida diretamente no código da aplicação. Em vez disso, seguiremos uma abordagem baseada em variáveis de ambiente, centralizada no módulo de configuração do projeto:

```
1 delivery/ext/config/__init__.py
```

Esse módulo é responsável por carregar variáveis do ambiente e registrá-las no objeto de configuração da aplicação. Para incluir o banco de dados, estendemos a função `init_app` com as seguintes definições:

```
1 app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get(  
2     'DATABASE_URL',  
3     'sqlite:///app.db'  
4 )  
5 app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

Nessa configuração, a URL do banco é obtida a partir da variável de ambiente `DATABASE_URL`. Caso essa variável não esteja definida, utiliza-se como padrão um banco SQLite local.

Essa estratégia apresenta duas vantagens importantes. Em primeiro lugar, permite alterar o banco de dados utilizado sem modificar o código da aplicação, bastando ajustar as variáveis de ambiente. Em segundo lugar, facilita a transição entre diferentes contextos de execução, como desenvolvimento local, testes e produção.

Dessa forma, a aplicação passa a depender apenas de uma interface de configuração abstrata, enquanto os detalhes concretos da infraestrutura permanecem externalizados.

Estrutura do Projeto e Inicialização

Considerando a estrutura do projeto, a inicialização do banco de dados será organizada no módulo dedicado a extensões:

```
1 delivery/ext/
```

Essa decisão segue um princípio arquitetural fundamental: extensões devem ser declaradas de forma independente da aplicação e inicializadas posteriormente. Esse padrão reduz o acoplamento entre componentes e permite maior flexibilidade, especialmente em cenários com múltiplas instâncias da aplicação, execução de testes ou uso de scripts auxiliares.

No módulo responsável pelo banco de dados (por exemplo, `delivery/ext/db/__init__.py`), definimos inicialmente o objeto da extensão:

Código: `delivery/ext/db/__init__.py`

```
1 from flask_sqlalchemy import SQLAlchemy
2
3 db = SQLAlchemy()
```

Nesse ponto, o objeto `db` ainda não está associado a nenhuma aplicação Flask. Essa associação ocorre por meio de uma função de inicialização, que recebe a aplicação como argumento:

```
1 def init_app(app):
2     db.init_app(app)
```

Esse padrão mantém a extensão desacoplada e permite que sua inicialização seja controlada explicitamente durante a criação da aplicação.

Um aspecto frequentemente negligenciado, mas essencial em projetos estruturados, é o registro dos modelos no *metadata* do SQLAlchemy. Como os modelos podem estar distribuídos em diferentes módulos do projeto, é necessário garantir que todos sejam importados antes de operações como criação de tabelas ou migrações.

Para isso, definimos uma função auxiliar responsável por carregar explicitamente os módulos de modelos:

```
1 def register_models():
2     """
3     Importa todos os modulos que definem modelos para que sejam registrados
4     no metadata do SQLAlchemy antes de operacoes como create_all().
5     """
6     import delivery.models.role
7     import delivery.models.user
8     import delivery.models.level
9     import delivery.models.business
10    import delivery.models.business_type
11    import delivery.models.role_user
12    import delivery.models.location
13    #import delivery.models.order
```

Embora essas importações possam parecer redundantes, elas garantem que o SQLAlchemy tenha conhecimento de todas as classes mapeadas antes de qualquer operação estrutural no banco de dados.

Essa abordagem se torna particularmente importante à medida que o projeto cresce e os modelos passam a ser organizados em múltiplos pacotes.

Em conjunto, essa estrutura estabelece um ciclo claro de inicialização: a extensão é declarada de forma independente, integrada à aplicação no momento adequado e, por fim, associada a todos os modelos definidos no sistema.

Esse padrão será utilizado nas próximas seções, quando introduzirmos as entidades do domínio como classes ORM.

Integração com a Aplicação Flask

A integração entre o banco de dados e a aplicação Flask ocorre durante o processo de criação da aplicação, seguindo o padrão conhecido como *application factory*. No projeto em desenvolvimento, essa lógica está centralizada no arquivo `app.py`, localizado na raiz.

A seguir, apresentamos a estrutura completa do método `create_app`, já incorporando a inicialização do banco de dados:

Código: app.py

```
1 import logging
2 from flask import Flask
3
4 def create_app(test_config=None):
5     app = Flask(__name__)
6
7     if app.debug:
8         app.logger.setLevel(logging.DEBUG)
9
10    # -----
11    # Configuracao da aplicacao (variaveis de ambiente)
12    # -----
13    from delivery.ext.config import init_app as init_config
14    init_config(app)
15
16    if test_config:
17        app.config.update(test_config)
18
19    # -----
20    # Inicializacao do banco de dados
21    # -----
22    from delivery.ext.db import init_app as init_db
23    init_db(app)
24
25    # Registro dos modelos no metadata do SQLAlchemy
26    from delivery.ext.db import register_models
27    register_models()
28
29    # -----
30    # Outras extensoes
31    # -----
32    if not app.config.get("TESTING"):
33        from delivery.ext.wtf import init_app as init_wtf
34        init_wtf(app)
35
36    from delivery.ext.debugtoolbar import init_app as init_toolbar
37    init_toolbar(app)
```

```
38
39 # -----
40 # Blueprints (camada de apresentacao)
41 # -----
42 from delivery.views import init_app as init_site
43 init_site(app)
44
45 return app
```

Observe que a inicialização do banco de dados ocorre após a configuração da aplicação. Isso é essencial, pois a extensão depende de parâmetros como a URL de conexão (SQLALCHEMY_DATABASE_URI), definidos previamente no módulo de configuração.

Além disso, a chamada à função `register_models()` garante que todas as classes ORM sejam carregadas antes de qualquer operação estrutural no banco de dados. Sem esse passo, o SQLAlchemy pode não reconhecer modelos definidos em módulos ainda não importados.

Esse fluxo evidencia uma característica central do padrão *application factory*: cada componente da aplicação — configuração, banco de dados, extensões e interface — é inicializado de forma explícita e controlada. Isso reduz acoplamento, melhora a organização do código e facilita a adaptação da aplicação a diferentes contextos, como testes automatizados ou ambientes de produção.

Considerações Arquiteturais

A adoção do Flask-SQLAlchemy neste projeto vai além de uma escolha voltada à conveniência de uso. Ela estabelece uma base consistente para integrar o modelo relacional, discutido anteriormente, à sua representação concreta em código, preservando tanto a estrutura dos dados quanto a expressividade do paradigma orientado a objetos.

Essa integração se apoia em uma decisão arquitetural central: tratar o banco de dados como uma extensão independente da aplicação. Na prática, isso se manifesta na separação entre três momentos distintos — configuração, inicialização e uso — cada um com responsabilidade bem definida dentro da estrutura do projeto.

A configuração é centralizada no módulo dedicado, que define parâmetros como a URL de conexão por meio de variáveis de ambiente. A inicialização ocorre explicitamente durante a criação da aplicação, quando a extensão é associada ao objeto Flask por meio da função `init_app`. Por fim, o uso do banco de dados é realizado nos módulos de domínio, onde as entidades são definidas como classes ORM e manipuladas ao longo do ciclo de execução da aplicação.

Essa separação permite maior controle sobre o ciclo de vida das conexões com o banco de dados e reduz o acoplamento entre componentes, facilitando a evolução do sistema. Além disso, garante compatibilidade com práticas amplamente adotadas em aplicações Flask de médio e grande porte, especialmente aquelas que utilizam o padrão *application factory*.

Outro aspecto importante dessa arquitetura é a necessidade de registrar explicitamente os modelos no *metadata* do SQLAlchemy. Esse passo assegura que todas as entidades definidas no sistema sejam reconhecidas antes de operações estruturais no banco de dados, como criação de tabelas ou execução de migrações.

Em conjunto, essas decisões estabelecem uma base sólida para a construção do sistema. Nas próximas seções, essa infraestrutura será utilizada para definir as entidades do domínio como classes ORM, consolidando a transição entre o modelo teórico e sua implementação concreta.

Definição de Modelos

No contexto de aplicações que utilizam ORM, o termo *modelo* refere-se às entidades persistentes do sistema, isto é, aquelas estruturas que são armazenadas no banco de dados e manipuladas ao longo da execução da aplicação.

Em Flask-SQLAlchemy, um modelo é tipicamente definido como uma classe Python que herda de `db.Model`. Cada atributo da classe corresponde a uma coluna da tabela associada, estabelecendo uma correspondência direta entre o modelo relacional e sua representação em código.

De forma conceitual, essa abordagem implementa o mapeamento discutido anteriormente:

$$\mathcal{M} : \text{Classes} \rightarrow \text{Relações}$$

no qual classes Python passam a representar tabelas, e objetos representam tuplas.

A estrutura básica de um modelo envolve três elementos principais: o nome da tabela, definido pela variável `__tablename__`, os atributos persistentes, definidos por colunas, e os relacionamentos com outras entidades.

A seguir, apresentamos a definição dos principais modelos do sistema.

Modelos do Sistema

Vamos adotar uma estrutura modular no projeto referente ao banco de dados, com cada entidade do domínio sendo definida em um arquivo próprio dentro do pacote `delivery/models`. Essa organização nos permitirá tratar cada modelo como uma unidade independente, facilitando tanto a manutenção quanto a compreensão da estrutura do sistema. Entretanto, precisamos garantir que todas as entidades definidas no sistema sejam corretamente reconhecidas pelo SQLAlchemy.

No contexto de um ORM, não basta que as classes estejam definidas em arquivos separados. Para que o sistema consiga, por exemplo, criar as tabelas no banco de dados ou estabelecer relacionamentos entre entidades, é necessário que todos os modelos estejam carregados em memória. Esse processo é conhecido como registro no *metadata* do SQLAlchemy.

Para garantir esse comportamento de forma controlada, utilizamos o arquivo `delivery/models/__init__.py` com o seguinte conteúdo:

Código: `delivery/models/__init__.py`

```
1 from .user import User
2 from .role import Role
3 from .level import Level
4 from .business import Business
5 from .business_type import BusinessType
6 from .role_user import RoleUser
```

```
7 from .location import Address, City
```

Esse arquivo exerce um papel central na arquitetura do sistema. Ao importar explicitamente cada modelo, asseguramos que todas as classes derivadas de `db.Model` sejam registradas no momento em que o pacote `delivery.models` é carregado.

Em termos práticos, isso significa que uma única instrução:

```
1 import delivery.models
```

é suficiente para tornar todas as entidades conhecidas pelo ORM. Essa abordagem evita a necessidade de múltiplas importações dispersas ao longo do código e garante que operações como **criação de tabelas**, **geração de migrações** e **resolução de relacionamentos entre modelos** funcionem corretamente.

Do ponto de vista arquitetural, essa estratégia estabelece um ponto único de agregação para o domínio da aplicação. Em vez de depender implicitamente da ordem de importação dos módulos, tornamos explícito quais entidades fazem parte do sistema.

Além disso, essa organização facilita a evolução do projeto. Novos modelos podem ser adicionados de forma incremental, bastando incluí-los nesse arquivo para que passem a integrar o conjunto de entidades gerenciadas pelo ORM.

Essa etapa, embora simples em aparência, é fundamental para o funcionamento consistente do sistema. Sem o registro adequado dos modelos, o SQLAlchemy não consegue construir corretamente o mapeamento entre o código e o banco de dados, comprometendo tanto a integridade estrutural quanto o comportamento da aplicação.

Aviso

Modelos não pertencem à camada de interface (views), mas sim ao núcleo do sistema.

Modelo Role

Iniciamos pela definição da entidade `Role`, responsável por representar os diferentes papéis que podem existir no sistema.

Código: delivery/models/role.py

```

1 from typing import List, TYPE_CHECKING
2 from sqlalchemy.orm import Mapped, mapped_column, relationship
3 from delivery.ext.db import db
4
5 if TYPE_CHECKING:
6     from .role_user import RoleUser
7
8 # === Role (papel) ===
9 class Role(db.Model):
10     __tablename__ = "roles"
11     __table_args__ = {'extend_existing': True}
12
13     id: Mapped[int] = mapped_column(db.Integer, primary_key=True)
14     name: Mapped[str] = mapped_column(db.String(30), unique=True, index=True)
15     status: Mapped[bool] = mapped_column(db.Boolean, default=True)
16
17     role_associations: Mapped[List["RoleUser"]] = relationship(
18         "RoleUser",
19         back_populates="role",
20         cascade="all, delete-orphan",
21     )
22
23     def __repr__(self) -> str:
24         return f"<Role {self.name}>"

```

A variável `__tablename__` define explicitamente o nome da tabela no banco de dados. Embora o SQLAlchemy possa gerar nomes automaticamente, essa definição evita ambiguidades e garante aderência às convenções adotadas no projeto.

Outro ponto importante é o uso de:

```
__table_args__ = {'extend_existing': True}
```

Esse parâmetro instrui o SQLAlchemy a permitir a redefinição de uma tabela já existente no *metadata*. Em aplicações Flask executadas em modo de desenvolvimento, especialmente com recarregamento automático (*debug mode*), um mesmo módulo pode ser importado mais de uma vez. Nessas situações, o parâmetro evita erros do tipo:

Do ponto de vista arquitetural, essa separação traz vantagens importantes como evitar duplicação de papéis, reduzir inconsistências semânticas, permitir reutilização do mesmo papel em diferentes empresas e facilitar expansão futura do sistema.

O relacionamento com a entidade `RoleUser` é definido por meio da função:

```
1 relationship()
```

Note que o nome da entidade associada é fornecido como string `"RoleUser"`. Essa técnica implementa uma resolução tardia (*lazy resolution*) da referência, evitando dependências circulares entre módulos Python.

Para preservar a consistência da tipagem sem provocar importações em tempo de execução, utilizamos:

```
1 from typing import TYPE_CHECKING
```

em conjunto com:

```
1 if TYPE_CHECKING:  
2     from .role_user import RoleUser
```

Esse padrão permite que ferramentas de análise estática reconheçam corretamente os tipos envolvidos, sem comprometer o desacoplamento entre os módulos da aplicação.

O relacionamento com a entidade `RoleUser` é definido por meio da função `relationship()`.

Observe a configuração utilizada:

```
1 role_associations: Mapped[List["RoleUser"]] = relationship(  
2     "RoleUser",  
3     back_populates="role",  
4     cascade="all, delete-orphan",  
5 )
```

O primeiro argumento `"RoleUser"` é fornecido como string para permitir que o SQLAlchemy resolva a referência apenas posteriormente, evitando dependências circulares entre módulos Python.

Já o parâmetro `back_populates="role"` estabelece uma associação bidirecional explícita entre as entidades. Isso significa que o atributo `role_associations` da classe `Role` está

sincronizado com o atributo `role` da entidade `RoleUser`.

Na prática, essa configuração garante coerência entre os objetos manipulados em memória. Por exemplo, ao associar uma instância de `RoleUser` a determinado papel, o SQLAlchemy atualiza automaticamente ambos os lados do relacionamento:

`Role ↔ RoleUser`

Sem o uso de `back_populates`, seria necessário manter essa sincronização manualmente, aumentando a complexidade e o risco de inconsistências no estado da aplicação.

Para preservar a consistência da tipagem sem provocar importações em tempo de execução, utilizamos:

```
1 from typing import TYPE_CHECKING
```

em conjunto com:

```
1 if TYPE_CHECKING:
2     from .role_user import RoleUser
```

Esse padrão permite que ferramentas de análise estática reconheçam corretamente os tipos envolvidos, sem comprometer o desacoplamento entre os módulos da aplicação.

Por fim, o parâmetro:

```
1 cascade="all, delete-orphan"
```

define o comportamento de propagação das operações entre entidades relacionadas.

O termo `all` indica que operações realizadas sobre objetos da entidade `Role` podem ser propagadas automaticamente para os objetos associados em `RoleUser`, incluindo inserções, atualizações, remoções e sincronizações de sessão.

Já `delete-orphan` garante que associações órfãs sejam removidas automaticamente. Em outras palavras, se um vínculo deixar de estar associado a um objeto pai válido, o SQLAlchemy elimina automaticamente esse registro da tabela associativa.

Essa estratégia reduz a possibilidade de inconsistências lógicas no banco de dados, evitando a permanência de associações sem contexto válido.

Dessa forma, a entidade `Role` deve ser interpretada como um catálogo global de papéis disponíveis no sistema. A semântica completa da associação — envolvendo usuário, empresa, nível hierárquico e temporalidade — será incorporada posteriormente pela entidade `RoleUser`, responsável por contextualizar operacionalmente cada vínculo existente no sistema.

Modelo User

A entidade `User` ocupa uma posição central no sistema, representando a identidade básica de qualquer ator da aplicação. Todas as demais especializações operacionais — como cliente, administrador, gerente ou entregador — serão derivadas dessa estrutura por meio da associação com papéis e contextos organizacionais.

Código: `delivery/models/user.py`

```

1 from typing import List, Optional, TYPE_CHECKING
2 from datetime import datetime
3 from sqlalchemy.orm import Mapped, mapped_column, relationship
4 from sqlalchemy import func
5 from delivery.ext.db import db
6
7 if TYPE_CHECKING:
8     from .role_user import RoleUser
9     from .location import Address
10
11 # === User (usuario) ===
12 class User(db.Model):
13     __tablename__ = "users"
14     __table_args__ = {'extend_existing': True}
15
16     id: Mapped[int] = mapped_column(db.Integer, primary_key=True)
17     name: Mapped[str] = mapped_column(db.String(100), index=True)
18     email: Mapped[str] = mapped_column(db.String(100), unique=True, index=True)
19     phone: Mapped[Optional[str]] = mapped_column(db.String(15))
20     email_verified_at: Mapped[Optional[datetime]] = mapped_column(db.DateTime(
21         timezone=True))
22     photo: Mapped[Optional[str]] = mapped_column(db.String(100))

```

```
22 password: Mapped[Optional[str]] = mapped_column(db.String(255))
23 google_id: Mapped[Optional[str]] = mapped_column(db.String(255))
24 facebook_id: Mapped[Optional[str]] = mapped_column(db.String(255))
25 cpf: Mapped[Optional[str]] = mapped_column(db.String(15), index=True)
26 created_at: Mapped[datetime] = mapped_column(db.DateTime(timezone=True),
        server_default=func.now())
27 updated_at: Mapped[Optional[datetime]] = mapped_column(db.DateTime(timezone=True)
        ), onupdate=func.now())
28 is_active: Mapped[bool] = mapped_column(db.Boolean, default=True, nullable=False
        , index=True)
29
30 role_associations: Mapped[List["RoleUser"]] = relationship(
31     "RoleUser",
32     back_populates="user",
33     cascade="all, delete-orphan",
34 )
35
36 addresses: Mapped[List["Address"]] = relationship(
37     "Address",
38     back_populates="user",
39     cascade="all, delete-orphan"
40 )
41
42 @property
43 def roles(self):
44     return [
45         assoc.role
46         for assoc in self.role_associations
47         if assoc.role
48     ]
49
50 def __repr__(self) -> str:
51     return f"<User {self.email}>"
```

A estrutura dessa entidade introduz diversos elementos importantes da modelagem ORM moderna. O primeiro aspecto relevante é a utilização intensiva de tipagem explícita por meio de `Mapped[...]`. Esse padrão, introduzido no SQLAlchemy 2.x, permite declarar de forma simultânea:

- o tipo Python esperado;
- a estrutura relacional correspondente;
- as restrições semânticas associadas ao atributo.

Por exemplo:

```
1 email: Mapped[str]
```

indica que o atributo deve armazenar uma cadeia de caracteres, enquanto:

```
1 phone: Mapped[Optional[str]]
```

define explicitamente que o valor pode ser nulo.

O uso recorrente de `Optional[...]` também merece atenção especial. Essa anotação indica que determinados campos não são obrigatórios do ponto de vista lógico ou operacional. Em sistemas reais, isso é particularmente importante porque muitas informações podem não estar disponíveis no momento inicial do cadastro.

Por exemplo, um usuário pode autenticar-se inicialmente apenas com conta Google, o CPF pode ainda não ter sido informado, a foto de perfil pode não existir e o telefone pode ser cadastrado posteriormente.

Essa flexibilidade, entretanto, deve ser utilizada com cautela. Permitir muitos valores nulos pode comprometer a consistência semântica do banco de dados e dificultar validações futuras.

Outro ponto importante do modelo é a presença de atributos temporais. O campo `created_at` utiliza `server_default=func.now()`, delegando ao próprio banco de dados a responsabilidade de registrar automaticamente o instante de criação da tupla.

Já `updated_at` utiliza `onupdate=func.now()`, permitindo atualização automática sempre que a linha sofre modificações.

Esses dois mecanismos representam estratégias distintas de controle temporal: (i) controle de inserção e (ii) controle de atualização.

Observe ainda que o modelo não possui qualquer atributo relacionado diretamente a níveis

hierárquicos, permissões ou cargos específicos. Essa decisão é intencional e decorre da modelagem conceitual adotada no sistema.

O usuário representa apenas a identidade da pessoa dentro da plataforma. Já as permissões operacionais dependem: i) do papel exercido, ii) da empresa associada, iii) do contexto organizacional, e iv) do nível hierárquico atribuído.

Em outras palavras, um mesmo usuário pode ser cliente global da plataforma, atuar como gerente de uma pizzeria, ser administrador de uma hamburgueria ou ainda possuir diferentes níveis em cada contexto.

Por essa razão, informações como nível e permissões não pertencem semanticamente à entidade `User`. Elas serão modeladas posteriormente na entidade associativa `RoleUser`, responsável por contextualizar os vínculos entre:

$$\text{User} \leftrightarrow \text{Role} \leftrightarrow \text{Business}$$

Do ponto de vista estrutural, a entidade apresenta dois relacionamentos principais. O primeiro `role_associations` estabelece uma ligação com a entidade associativa `RoleUser`. Trata-se de um relacionamento muitos-para-muitos enriquecido com atributos próprios, permitindo incorporar: i) contexto empresarial, ii) nível hierárquico, iii) temporalidade, e iv) histórico de vínculos.

O segundo relacionamento `addresses` define uma associação do tipo um-para-muitos entre usuários e endereços. Nesse caso, um único usuário pode possuir múltiplos registros de endereço associados ao seu perfil.

Em ambos os relacionamentos, utilizamos `back_populates` para garantir sincronização bidirecional entre as entidades relacionadas. Isso significa que alterações realizadas em um lado da associação são refletidas automaticamente no outro lado do relacionamento ORM.

Além disso, o parâmetro `cascade="all, delete-orphan"` define regras automáticas de propagação de operações. Na prática, inserções podem ser propagadas, atualizações podem ser sincronizadas, remoções podem ser encadeadas e associações órfãs são eliminadas automaticamente. Essa estratégia reduz a possibilidade de inconsistências lógicas e evita a permanência de registros sem contexto válido no banco de dados.

Outro elemento importante do modelo é a propriedade:

```
1 @property
2 def roles(self):
```

Essa construção introduz uma camada adicional de abstração sobre o relacionamento ORM. Em vez de expor diretamente toda a estrutura intermediária representada por `RoleUser` a propriedade permite acessar apenas os papéis efetivamente associados ao usuário.

Assim, em vez de `user.role_associations` podemos utilizar `user.roles`. Esse tipo de abstração é extremamente comum em ORMs modernos, pois permite encapsular detalhes da modelagem relacional e oferecer interfaces mais expressivas para o restante da aplicação.

Dessa forma, a entidade `User` deve ser interpretada como o núcleo identitário do sistema. Ela concentra informações pessoais e mecanismos básicos de autenticação, enquanto as responsabilidades operacionais e organizacionais serão contextualizadas posteriormente pelas entidades associativas do modelo.

NOTA

Note que repetimos explicitamente diversas importações em cada módulo. Em Python, imports explícitos também funcionam como documentação arquitetural, tornando cada arquivo autocontido e semanticamente mais claro.

Modelo Level

A entidade `Level` é responsável por representar níveis hierárquicos associados aos vínculos entre usuários, papéis e empresas no sistema.

Diferentemente da entidade `Role`, cuja função é apenas descrever quais papéis existem na plataforma, a entidade `Level` introduz uma camada adicional de qualificação operacional. Em outras palavras, enquanto `Role` responde à pergunta:

“qual papel o usuário exerce?”

a entidade `Level` complementa essa informação indicando:

“qual é o nível hierárquico desse papel?”

Essa separação possui implicações arquiteturais importantes. Em sistemas reais, dois usuários podem exercer exatamente o mesmo papel dentro de uma mesma empresa, mas com níveis distintos de autoridade. Por exemplo:

- dois administradores podem possuir poderes diferentes;
- dois atendentes podem operar com permissões distintas;
- um cliente pode pertencer a categorias especiais da plataforma.

Ao isolar a noção de nível em uma entidade própria, o sistema ganha maior flexibilidade semântica e reduz acoplamentos indevidos entre identidade, papel e hierarquia.

Código: `delivery/models/level.py`

```
1 from typing import List, TYPE_CHECKING
2 from sqlalchemy.orm import Mapped, mapped_column, relationship
3 from delivery.ext.db import db
4
5 if TYPE_CHECKING:
6     from .role_user import RoleUser
7
8 # === Level (nível hierárquico) ===
9 class Level(db.Model):
10     __tablename__ = "levels"
11     __table_args__ = {'extend_existing': True}
12
13     id: Mapped[int] = mapped_column(db.Integer, primary_key=True)
14     name: Mapped[str] = mapped_column(db.String(30), unique=True, nullable=False,
15                                     index=True)
16     description: Mapped[str | None] = mapped_column(db.String(255))
17     status: Mapped[bool] = mapped_column(db.Boolean, default=True, nullable=False)
18
19     role_associations: Mapped[List["RoleUser"]] = relationship(
20         "RoleUser",
```

```

20     back_populates="level"
21 )
22
23 def __repr__(self) -> str:
24     return f"<Level {self.name}>"

```

Assim como nos modelos anteriores, a variável `__tablename__` define explicitamente o nome da tabela relacional correspondente. Já o parâmetro `__table_args__ = {'extend_existing': True}` mantém compatibilidade com o fluxo de desenvolvimento iterativo adotado no projeto.

O atributo `name` define o nome lógico do nível hierárquico. Inicialmente, o sistema trabalhará com quatro níveis principais: God¹, Premium, Better e Standard.

Embora esses nomes possam parecer associados apenas a clientes, a modelagem foi construída de forma genérica. Isso significa que os níveis poderão futuramente representar hierarquias administrativas, categorias operacionais ou políticas de permissão distintas dentro de empresas específicas.

O atributo `description` permite documentar semanticamente o significado de cada nível, favorecendo manutenção futura e maior clareza organizacional.

Já o atributo `status` possibilita ativar ou desativar níveis sem necessidade de removê-los fisicamente do banco de dados. Essa abordagem preserva integridade histórica e evita perda de vínculos previamente registrados.

O relacionamento:

```

1 role_associations: Mapped[List["RoleUser"]] = relationship(
2     "RoleUser",
3     back_populates="level"
4 )

```

estabelece uma associação bidirecional entre `Level` e `RoleUser`. Dessa forma, cada vínculo operacional pode possuir exatamente um nível associado, enquanto um mesmo nível pode ser reutilizado por múltiplas associações distintas.

¹Sim... é uma piada!

Observe novamente o uso de `back_populates="level"`. Esse parâmetro sincroniza automaticamente ambos os lados do relacionamento ORM:

`Level ↔ RoleUser`

Na prática, isso garante consistência entre os objetos carregados em memória e reduz a necessidade de sincronizações manuais entre entidades relacionadas.

Do ponto de vista conceitual, a criação da entidade `Level` representa uma importante evolução da modelagem. Em vez de armazenar níveis diretamente em `User` ou mesmo em `Role`, o sistema passa a tratar hierarquia como uma característica contextual do vínculo operacional existente entre:

`User ↔ Role ↔ Business`

Essa decisão torna o modelo significativamente mais flexível, permitindo que um mesmo usuário exerça o mesmo papel em empresas distintas, com níveis hierárquicos independentes em cada contexto organizacional.

Modelo `BusinessType`

A entidade `BusinessType` é responsável por representar os diferentes tipos de estabelecimentos cadastrados na plataforma.

Em sistemas de delivery, comércio eletrônico ou marketplaces, é bastante comum que diferentes empresas operem sob categorias distintas. Por exemplo: i) pizzeria, ii) hamburgueria, iii) restaurante, iv) padaria, v) açaiateria, vi) cafeteria, vii) food truck, viii) marmitaria, e ix) sorveteria.

Embora todas essas organizações sejam tratadas genericamente como empresas dentro da plataforma, cada categoria possui características operacionais próprias. Em muitos cenários reais, determinadas funcionalidades podem estar disponíveis apenas para alguns tipos específicos de negócio.

Separar essa informação em uma entidade própria reduz redundâncias e melhora significativamente a flexibilidade da modelagem. O código a seguir apresenta nossa implementação deste modelo.

Código: delivery/models/business_type.py

```

1 from typing import List, Optional, TYPE_CHECKING
2 from sqlalchemy.orm import Mapped, mapped_column, relationship
3 from delivery.ext.db import db
4
5 if TYPE_CHECKING:
6     from .business import Business
7
8 class BusinessType(db.Model):
9     __tablename__ = "business_types"
10    __table_args__ = {'extend_existing': True}
11
12    id: Mapped[int] = mapped_column(db.Integer, primary_key=True)
13    name: Mapped[str] = mapped_column(db.String(80), unique=True, nullable=False,
14                                     index=True)
15    description: Mapped[Optional[str]] = mapped_column(db.String(255))
16    status: Mapped[bool] = mapped_column(db.Boolean, default=True, nullable=False)
17
18    businesses: Mapped[List["Business"]] = relationship(
19        "Business",
20        back_populates="business_type"
21    )
22
23    def __repr__(self) -> str:
24        return f"<BusinessType {self.name}>"

```

Assim como nos modelos anteriores, a variável `__tablename__` define explicitamente o nome da tabela relacional associada ao modelo. Já a configuração:

```

1 __table_args__ = {'extend_existing': True}

```

mantém compatibilidade com recriações sucessivas das tabelas durante o processo iterativo de desenvolvimento.

O atributo `name: Mapped[str]` define o nome lógico do tipo de estabelecimento. A combinação dos parâmetros `unique=True`, `nullable=False` e `index=True` impõe três propriedades respectivamente importantes ao modelo: (a) o nome do tipo não pode ser nulo, (b) não podem existir dois tipos com o mesmo nome, e (c) consultas baseadas nesse

atributo tornam-se mais eficientes.

Na prática, isso evita inconsistências como múltiplos registros duplicados para categorias equivalentes:

`"Pizzaria" e "pizzaria"`

O atributo `description: Mapped[Optional[str]]` permite documentar semanticamente cada categoria cadastrada. Embora opcional, esse campo favorece manutenção futura, auditoria administrativa e maior clareza organizacional dentro do sistema.

Já o atributo `status: Mapped[bool]` permite ativar ou desativar tipos de negócio sem removê-los fisicamente do banco de dados.

Essa abordagem é especialmente importante em aplicações reais. Em vez de apagar registros históricos — o que poderia comprometer integridade referencial — o sistema apenas marca determinados tipos como inativos:

`status = False`

O relacionamento:

```
1 businesses: Mapped[List["Business"]] = relationship(  
2     "Business",  
3     back_populates="business_type"  
4 )
```

estabelece uma associação bidirecional entre `BusinessType` e `Business`.

Conceitualmente, isso significa que um tipo de negócio pode estar associado a várias empresas e que cada empresa pertence a exatamente um tipo de negócio.

Em termos relacionais:

`BusinessType 1 : N Business`

O parâmetro `back_populates="business_type"`, como já discutido, sincroniza automaticamente ambos os lados do relacionamento ORM:

`BusinessType ↔ Business`

Dessa forma, ao carregar um objeto `BusinessType`, o SQLAlchemy poderá recuperar automaticamente todas as empresas associadas àquela categoria. Analogamente, cada objeto `Business` poderá acessar diretamente o seu tipo correspondente.

Finalmente, o método:

```
1 def __repr__(self) -> str:
2     return f"<BusinessType {self.name}>"
```

define uma representação textual amigável do objeto, facilitando depuração, inspeção no terminal e análise de logs durante o desenvolvimento da aplicação.

Modelo Business

A entidade `Business` foi concebida para representar genericamente qualquer unidade organizacional ou estabelecimento participante da plataforma. Sua implementação é fundamental para suportar o cenário onde um mesmo usuário interage com o ecossistema de múltiplas formas. Na prática, um usuário pode:

- ser proprietário de uma unidade (ex: uma hamburgueria);
- atuar como gerente ou colaborador em outros estabelecimentos (ex: uma pizzaria);
- utilizar a aplicação simultaneamente como cliente final, realizando pedidos em diversas frentes.

O modelo `Business` viabiliza uma arquitetura *multi-tenant* (multi-empresa), na qual os vínculos de permissões (`RoleUser`) não são globais, mas estritamente restritos ao contexto de uma unidade de negócio. Assim, a estrutura lógica evolui para o seguinte núcleo central:

User $\longleftrightarrow$ Business $\longleftrightarrow$ RoleUser $\longleftrightarrow$ Role

Diferente da tabela `User`, que concentra os dados da pessoa física (CPF), a `Business` armazena os dados da pessoa jurídica (CNPJ, Razão Social) e possui sua própria localização física.

O código a seguir apresenta a implementação atualizada do modelo Business.

Código: delivery/models/business.py

```
1 from typing import List, Optional, TYPE_CHECKING
2 from datetime import datetime
3 from sqlalchemy.orm import Mapped, mapped_column, relationship
4 from sqlalchemy import func, ForeignKey, Integer, String, Boolean, DateTime
5 from delivery.ext.db import db
6
7 if TYPE_CHECKING:
8     from .user import User
9     from .role_user import RoleUser
10    from .business_type import BusinessType
11    from .location import Address
12
13 class Business(db.Model):
14     __tablename__ = "businesses"
15     __table_args__ = {'extend_existing': True}
16
17     id: Mapped[int] = mapped_column(Integer, primary_key=True)
18     owner_user_id: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False, index=True)
19     corporate_name: Mapped[str] = mapped_column(String(120), nullable=False)
20     trade_name: Mapped[str] = mapped_column(String(120), nullable=False, index=True)
21     cnpj: Mapped[str] = mapped_column(String(20), unique=True, nullable=False, index=True)
22     business_type_id: Mapped[int] = mapped_column(ForeignKey("business_types.id"), nullable=False, index=True)
23     description: Mapped[Optional[str]] = mapped_column(String(255))
24     is_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False, index=True)
25     created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
26
27     # Relacionamento com o Proprietario
28     owner: Mapped["User"] = relationship("User")
29
30     # Relacionamento com Endereco (1:1)
31     # O cascade garante a integridade: se a empresa for removida, o endereco tambem
```



```

32     sera.
33     address: Mapped[Optional["Address"]] = relationship(
34         "Address",
35         back_populates="business",
36         cascade="all, delete-orphan",
37         uselist=False
38     )
39
40     role_associations: Mapped[List["RoleUser"]] = relationship(
41         "RoleUser",
42         back_populates="business",
43         cascade="all, delete-orphan"
44     )
45
46     business_type: Mapped["BusinessType"] = relationship(
47         "BusinessType",
48         back_populates="businesses"
49     )
50
51     def __repr__(self) -> str:
52         return f"<Business {self.trade_name}>"

```

A configuração 'extend_existing': True em __table_args__ é mantida para facilitar o desenvolvimento iterativo, permitindo que o SQLAlchemy atualize a definição do modelo em memória sem conflitos durante recarregamentos automáticos.

O atributo owner_user_id estabelece a responsabilidade estrutural. Ele identifica o usuário “dono” da conta empresarial na plataforma. É vital notar que ser o owner define a posse da entidade jurídica, mas não substitui o mecanismo de autorização: as permissões operacionais do proprietário (e de seus funcionários) dentro do sistema continuam sendo geridas pela entidade associativa RoleUser. Essa separação permite, por exemplo, que um proprietário delegue toda a gestão operacional a um gerente sem perder a titularidade da empresa.

Normalização e Tipificação: a introdução da entidade BusinessType resolve o problema da falta de padronização. Armazenar o tipo de negócio como texto livre (ex: "Pizza-

ria”) resultaria em inconsistências de grafia e dificuldades em filtros de busca. Com a normalização:

$$\text{Business.business_type_id} \longrightarrow \text{BusinessType.id}$$

garantimos integridade referencial e facilitamos a expansão do sistema (como adicionar ícones ou regras específicas por categoria de negócio).

Neste ponto, apresentamos e reforçamos a distinção entre `ForeignKey` e `relationship`:

- `ForeignKey`: atua no nível do SGBD², impondo restrições físicas e garantindo que o `business_type_id` aponte para um registro existente.
- `relationship`: atua na camada do ORM³, permitindo a navegação orientada a objetos (ex: `empresa.business_type.name`).

O Papel no Modelo de Autorização: a conexão com `RoleUser` através de `role_associations` transforma radicalmente a semântica de autorização. O sistema deixa de validar apenas se o usuário “é um gerente” para validar se o usuário “é um gerente **nesta** empresa específica”.

A unicidade do vínculo de permissão agora é composta pela tríade:

$$(\text{user_id}, \text{role_id}, \text{business_id})$$

Essa estrutura permite que um usuário possua múltiplos papéis em diferentes contextos organizacionais simultaneamente. Adicionalmente, o modelo prevê que usuários operando apenas como clientes finais possuam um `business_id` nulo em seus vínculos de papel, mantendo a flexibilidade para operações globais na plataforma.

Por fim, a inclusão do relacionamento `address` permite que cada estabelecimento possua um endereço físico independente do endereço residencial de seu proprietário, essencial para a logística de entrega e exibição em mapas.

²SGBD é a sigla para Sistema de Gerenciamento de Banco de Dados (em inglês, DBMS - *Database Management System*).

³Lembrando que ORM significa *Object-Relational Mapping* (Mapeamento Objeto-Relacional).

Modelo RoleUser

A entidade RoleUser representa o núcleo semântico do sistema de autorização da aplicação. Embora estruturalmente funcione como uma tabela associativa, seu papel no modelo transcende a simples conexão entre usuários e papéis. É nesta entidade que o sistema contextualiza operacionalmente a relação quadridimensional:

User $\longleftrightarrow$ Role $\longleftrightarrow$ Business $\longleftrightarrow$ Level

Diferente de modelos de permissão simplistas, a RoleUser define com precisão:

- **Sujeito:** a qual usuário exerce determinado papel;
- **Escopo:** em qual unidade de negócio (*Business*) esse papel é aplicado;
- **Graduação:** qual o nível hierárquico (*Level*) associado ao vínculo;
- **Temporalidade:** o período de validade (ciclo de vida) desse vínculo no sistema.

Essa abordagem permite representar cenários organizacionais complexos, onde a autorização não é uma propriedade estática do usuário, mas um estado dependente do contexto.

Código: delivery/models/role_user.py

```

1 from typing import Optional, TYPE_CHECKING
2 from datetime import datetime
3 from sqlalchemy.orm import Mapped, mapped_column, relationship
4 from sqlalchemy import func, UniqueConstraint
5
6 from delivery.ext.db import db
7
8 if TYPE_CHECKING:
9     from .role import Role
10    from .user import User
11    from .business import Business
12    from .level import Level
13
14 # === Associacao entre usuario, papel e empresa ===

```

```
15 class RoleUser(db.Model):
16     __tablename__ = "roles_has_users"
17     __table_args__ = (
18         UniqueConstraint(
19             "user_id",
20             "role_id",
21             "business_id",
22             name="uq_user_role_business"
23         ),
24         {'extend_existing': True}
25     )
26
27     id: Mapped[int] = mapped_column(db.Integer, primary_key=True)
28     role_id: Mapped[int] = mapped_column(db.ForeignKey("roles.id"), nullable=False,
29                                         index=True)
29     user_id: Mapped[int] = mapped_column(db.ForeignKey("users.id"), nullable=False,
30                                         index=True)
30     business_id: Mapped[Optional[int]] = mapped_column(db.ForeignKey("businesses.id"),
31                                                         nullable=True, index=True)
31     level_id: Mapped[Optional[int]] = mapped_column(db.ForeignKey("levels.id"),
32                                                         nullable=True)
32     created_at: Mapped[datetime] = mapped_column(db.DateTime(timezone=True),
33                                                  server_default=func.now(), nullable=False)
33     finished_at: Mapped[Optional[datetime]] = mapped_column(db.DateTime(timezone=
34                                                                True))
34
35     role: Mapped["Role"] = relationship(
36         "Role",
37         back_populates="role_associations"
38     )
39     user: Mapped["User"] = relationship(
40         "User",
41         back_populates="role_associations"
42     )
43     business: Mapped[Optional["Business"]] = relationship(
44         "Business",
45         back_populates="role_associations"
46     )
47     level: Mapped[Optional["Level"]] = relationship(
```

```

48         "Level",
49         back_populates="role_associations"
50     )
51
52     def __repr__(self) -> str:
53         return (
54             f"<RoleUser "
55             f"user_id={self.user_id} "
56             f"role_id={self.role_id} "
57             f"business_id={self.business_id}>"
58         )

```

Decisões de Design de Banco de Dados: o uso de chaves primárias compostas por múltiplas chaves estrangeiras (*Composite Primary Keys*) tende a elevar a complexidade das consultas e dificultar o mapeamento em ORMs, além de engessar futuras migrações. Por essa razão, optou-se pela introdução de uma **Chave Primária Artificial** (*id*), que identifica unicamente cada vínculo.

Para garantir a integridade lógica sem abdicar da simplicidade da chave artificial, aplicou-se uma restrição de unicidade (*UniqueConstraint*) sobre a tríade:

$$(user_id, role_id, business_id) \longrightarrow \text{Vínculo Único}$$

Esta restrição garante que um mesmo usuário não possua duplicidade do mesmo papel dentro de uma mesma unidade de negócio, ao mesmo tempo que permite que ele exerça o mesmo papel (ex: "Gerente") em empresas distintas.

Opcionalidade do Contexto Empresarial: a definição de *business_id* como opcional (*nullable=True*) é uma decisão arquitetural estratégica. Ela permite que o sistema suporte papéis globais (como "Administrador do Sistema" ou "Cliente Final") que operam fora do escopo de um *tenant* específico, coexistindo harmonicamente com papéis estritamente organizacionais (como "Operador de Caixa").

Hierarquia e Temporalidade: diferente de modelos onde o "nível" é um simples atributo numérico no usuário, aqui o *level_id* é uma entidade própria. Isso permite o desacoplamento das permissões: o nível não pertence ao usuário ($level \not\subseteq User$), mas sim à **relação** que ele mantém com a empresa naquele momento.

Finalmente, os atributos `created_at` e `finished_at` transformam o registro em um dado histórico. Ao invés de deletar um vínculo (o que causaria perda de rastreabilidade), o sistema utiliza o conceito de *Soft Close* através do preenchimento de `finished_at`. O vínculo é considerado ativo apenas no intervalo:

$$t \in [\text{created_at}, \text{finished_at}) \text{ onde } \text{finished_at} = \text{NULL}$$

Do ponto de vista de implementação ORM, o uso de `back_populates` em todos os relacionamentos garante que a sincronização em memória seja mantida, permitindo que a partir de um objeto `Business` possamos acessar instantaneamente sua lista de colaboradores e seus respectivos papéis e níveis.

Modelos City e Address

As entidades `City` e `Address` introduzem a camada de persistência geoespacial do sistema. Diferentemente dos modelos de identidade ou permissões, esta subestrutura foca na organização hierárquica e na flexibilidade de vínculo, permitindo que a localização seja um atributo aplicável tanto a pessoas físicas (`User`) quanto a unidades organizacionais (`Business`).

Código: `delivery/models/location.py`

```
1 from typing import List, Optional, TYPE_CHECKING
2 from sqlalchemy.orm import Mapped, mapped_column, relationship
3 from sqlalchemy import ForeignKey, Integer, String, Index
4 from delivery.ext.db import db
5
6 if TYPE_CHECKING:
7     from .user import User
8     from .business import Business
9
10 # === City (cidade) ===
11 class City(db.Model):
12     __tablename__ = "cities"
13
14     __table_args__ = (
15         Index(
```

```

16         "idx_city_name_state_country",
17         "name", "state", "country"
18     ),
19     {'extend_existing': True}
20 )
21
22 id: Mapped[int] = mapped_column(Integer, primary_key=True)
23 name: Mapped[str] = mapped_column(String(100), nullable=False)
24 state: Mapped[Optional[str]] = mapped_column(String(2))
25 country: Mapped[Optional[str]] = mapped_column(String(50))
26 region: Mapped[Optional[str]] = mapped_column(String(50))
27
28 addresses: Mapped[List["Address"]] = relationship(
29     "Address",
30     back_populates="city"
31 )
32
33 def __repr__(self) -> str:
34     return f"<City {self.name}-{ ' - ' + self.state if self.state else ''}>"
35
36
37 # === Address (endereco) ===
38 class Address(db.Model):
39     __tablename__ = "address"
40     __table_args__ = {'extend_existing': True}
41
42     id: Mapped[int] = mapped_column(Integer, primary_key=True)
43     road: Mapped[Optional[str]] = mapped_column(String(100))
44     number: Mapped[Optional[int]] = mapped_column(Integer)
45     district: Mapped[Optional[str]] = mapped_column(String(100))
46     zipcode: Mapped[Optional[str]] = mapped_column(String(15))
47
48     # user_id opcional: permite enderecos vinculados a empresas
49     user_id: Mapped[Optional[int]] = mapped_column(
50         Integer,
51         ForeignKey("users.id", ondelete="CASCADE"),
52         nullable=True,
53         index=True
54     )

```

```
55
56 # business_id opcional: permite enderecos vinculados a usuarios
57 business_id: Mapped[Optional[int]] = mapped_column(
58     Integer,
59     ForeignKey("businesses.id", ondelete="CASCADE"),
60     nullable=True,
61     index=True
62 )
63
64 city_id: Mapped[int | None] = mapped_column(
65     Integer,
66     ForeignKey("cities.id", ondelete="SET NULL"),
67     nullable=True
68 )
69
70 user: Mapped[Optional["User"]] = relationship("User", back_populates="addresses"
71 )
72 business: Mapped[Optional["Business"]] = relationship("Business", back_populates
73     ="address")
74 city: Mapped["City"] = relationship("City", back_populates="addresses")
75
76 def __repr__(self) -> str:
77     return f"<Address {self.road}, {self.number} - {self.district}>"
```

A entidade City exemplifica o uso de **Índices Compostos** para otimização de performance. O índice `idx_city_name_state_country` é fundamental pois, em bancos de dados relacionais, nomes de cidades não são candidatos ideais a chaves únicas globais. A indexação conjunta de nome, estado e país reduz drasticamente o custo computacional de buscas geográficas e reforça a precisão da identificação no contexto da aplicação.

Já a entidade Address atua como um nó de ligação versátil. Uma alteração central neste modelo foi tornar as chaves estrangeiras `user_id` e `business_id` como opcionais (`nullable=True`). Esta decisão de design permite que a mesma tabela suporte dois fluxos distintos:

- **Fluxo B2C:** Endereços de entrega vinculados diretamente ao perfil do consumidor;

- **Fluxo B2B:** Endereços de sede ou filiais vinculados ao estabelecimento comercial.

Políticas de Integridade Referencial: o modelo utiliza duas estratégias de `ondelete` no SGBD que merecem análise técnica:

1. **CASCADE (em `user_id` e `business_id`):** define uma dependência existencial. Se o “pai” (Usuário ou Empresa) é removido, o “filho” (Endereço) perde sua razão de existir e é eliminado automaticamente. Isso previne o acúmulo de registros órfãos e lixo lógico no banco.
2. **SET NULL (em `city_id`):** define uma associação fraca. Se uma cidade for excluída do sistema, não desejamos perder os dados brutos do endereço (rua, número, bairro). O SGBD apenas limpa o vínculo, preservando a informação textual para auditoria ou recategorização posterior.

É importante que o aluno de computação distinga estas políticas de *Constraint* (SGBD) das políticas de *Cascade* do ORM. Enquanto o `ondelete` é uma instrução de integridade física no esquema do banco, o `cascade` do SQLAlchemy (como o `delete-orphan` usado em *Business*) é uma automação lógica que gerencia o estado dos objetos em memória durante a sessão.

Consolida-se, assim, uma estrutura de localização robusta e extensível, capaz de suportar a evolução do sistema multi-empresa sem a necessidade de duplicar tabelas de endereço para cada nova entidade que surja no domínio.

Modelo Lógico do Sistema (Notação Crow's Foot)

A representação a seguir utiliza a notação *Crow's Foot*⁴, amplamente empregada na modelagem lógica de bancos de dados relacionais. Nessa abordagem, as cardinalidades

⁴O termo **Crow's Foot** (Pé de Galinha) refere-se à representação gráfica das cardinalidades “muitos” (*N*) no diagrama. O símbolo, composto por três linhas que se ramificam a partir de um único ponto em direção à entidade, assemelha-se à marca deixada pela pata de uma ave no solo. Esta notação foi introduzida originalmente por Gordon Everest em 1976, sob o nome *Inverted Arrow*, mas popularizou-se na literatura de Engenharia de Software devido à sua semelhança visual com a pegada de um corvo.

são representadas diretamente nas conexões entre entidades, permitindo interpretar com clareza as dependências e restrições do modelo.

O refinamento apresentado nesta etapa incorpora uma mudança estrutural importante: os vínculos entre usuários e papéis passam agora a ser contextualizados por empresa (*Business*). Além disso, os níveis hierárquicos deixam de ser tratados como atributos simples e passam a constituir uma entidade própria do sistema.

A principal evolução deste refinamento consiste na contextualização organizacional dos papéis exercidos pelos usuários. No domínio da aplicação, um mesmo usuário pode atuar simultaneamente em diferentes empresas, desempenhando inclusive o mesmo papel em cada uma delas. Por exemplo, um usuário pode ser: (a) gerente de uma pizzeria, (b) administrador de uma hamburgueria, (c) operador de um mercado e (d) cliente comum da plataforma.

Observe que o significado operacional do papel depende não apenas do usuário e do papel em si, mas também da empresa associada ao vínculo. Formalmente, a identidade lógica da associação passa a ser dada por:

$$(User, Role, Business)$$

Essa mudança introduz uma separação semântica muito mais consistente para o domínio.

A entidade *Business* torna o sistema significativamente mais genérico e aderente a cenários reais, permitindo representar diferentes tipos de estabelecimentos comerciais, como: (i) Pizzaria, (ii) Hamburgueria, (iii) Restaurante, (iv) Padaria, (v) Açáteria, (vi) Cafeteria, (vii) Food Truck, (viii) Marmitaria e (ix) Sorveteria.

Do ponto de vista conceitual, a entidade *Business* representa a identidade jurídica e operacional do estabelecimento dentro da plataforma.

A modelagem adota uma chave artificial *id* para a entidade *RoleUser* como identificador primário da entidade associativa. Essa decisão simplifica relacionamentos futuros, melhora integrações com APIs e facilita manutenção do sistema. A unicidade semântica da associação deixa então de depender da chave primária e passa a ser garantida por restrições específicas (*UniqueConstraint*) definidas na implementação física do banco.

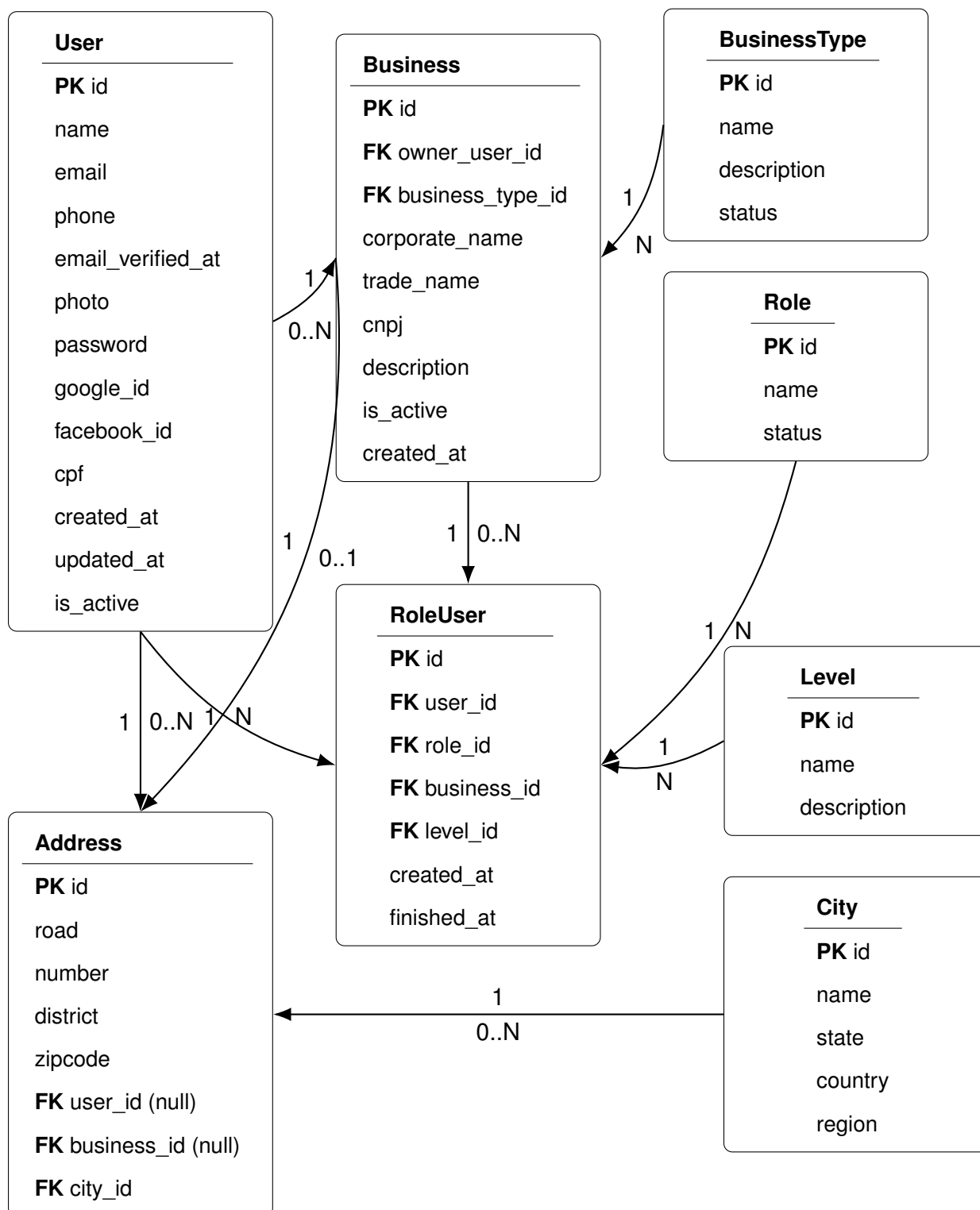


Figura 7.1: Modelo lógico em notação Crow's Foot do sistema de gerenciamento de usuários, papéis, empresas, categorias de negócio, níveis hierárquicos e estrutura de localização.

A entidade `Level` foi construída como uma entidade própria do sistema. Essa decisão melhora significativamente a padronização dos níveis, a consistência semântica, a reutilização e a flexibilidade evolutiva.

Formalmente, temos:

$$RoleUser \rightarrow Level$$

Isso significa que cada vínculo operacional possui exatamente um nível associado. Ao mesmo tempo, um mesmo nível pode ser reutilizado em múltiplos vínculos:

$$Level \rightarrow \{RoleUser_1, RoleUser_2, \dots\}$$

Caracterizando um relacionamento 1:N. Inicialmente, adotaremos quatro níveis hierárquicos:

Nível	Descrição Geral
God	maior nível hierárquico
Premium	nível intermediário superior
Basic	nível intermediário básico
Standard	nível padrão do sistema

Essa modelagem torna possível representar cenários como:

- proprietário de empresa com nível `Basic`;
- gerente operacional com nível `Premium`;
- atendente com nível `Basic`;
- cliente comum com nível `Standard`.

A entidade `RoleUser` passa então a atuar como o núcleo semântico da autorização do sistema. Ela deixa de ser apenas uma tabela de junção e passa a representar um vínculo operacional completo envolvendo identidade do usuário, papel exercido, empresa associada, nível hierárquico e validade temporal da associação.

Os atributos `created_at` e `finished_at` introduzem uma dimensão temporal explícita, permitindo interpretar cada associação como um intervalo:

$$[created_at, finished_at)$$

A ausência de valor em `finished_at` indica que o vínculo permanece ativo.

No subsistema de localização, a modelagem permanece baseada em duas entidades: `City` e `Address`. Nesse contexto:

- um usuário pode possuir múltiplos endereços;
- cada endereço pertence obrigatoriamente a um usuário;
- uma cidade pode estar associada a zero ou muitos endereços;
- a associação entre endereço e cidade é opcional.

Com esse refinamento, o modelo lógico torna-se significativamente mais aderente a cenários reais de plataformas multiempresa.

A separação entre identidade do usuário, papel exercido, empresa associada, hierarquia operacional e temporalidade fortalece o desacoplamento semântico do modelo e amplia sua capacidade de evolução futura.

Manipulação e Persistência de Dados

Com a definição das classes de modelo, o próximo estágio consiste em materializar a estrutura lógica no SGBD e compreender a mecânica de persistência do SQLAlchemy. Esta etapa representa a ponte entre a orientação a objetos e o modelo relacional, onde o ORM atua como um tradutor de estados.

No ecossistema Flask-SQLAlchemy, a interface primordial é o objeto `db.session`, que implementa o padrão de projeto *Unit of Work* (Unidade de Trabalho). Suas responsabilidades incluem:

- **Identity Map:** Garantir que cada registro do banco tenha apenas uma instância correspondente em memória.
- **Change Tracking:** Rastrear quais atributos de quais objetos foram modificados.
- **Transaction Management:** Coordenar o início e o fim das transações SQL.

Materialização do Esquema Físico

O primeiro passo para testar os modelos é gerar as tabelas no banco de dados. Utilizaremos o *Flask Shell* para interagir com o contexto da aplicação:

```
1 (venv) $ flask shell
```

No interpretador, executamos:

```
>>> from delivery.ext.db import db
>>> import delivery.models # Este comando "acorda" todos os ↵
    ↵ modelos no seu __init__.py
>>> db.create_all()
```

A instrução `db.create_all()` emite comandos `CREATE TABLE` para todas as classes que herdam de `db.Model`. É fundamental compreender que este método é **idempotente** e **não-destrutivo**: ele ignora tabelas que já existem. Para refletir mudanças estruturais durante o desenvolvimento, utilizamos o ciclo:

```
>>> db.drop_all() # Destrutivo: remove tabelas e dados
>>> db.create_all() # Recria conforme a definicao atual do ↵
    ↵ código
```

O Ciclo de Vida da Transação

O SQLAlchemy utiliza uma estratégia de *Flush* tardio. As alterações feitas em objetos Python ficam em um estado “pendente” na sessão. A persistência física só ocorre no `commit`. Formalmente, uma transação garante as propriedades **ACID**⁵, onde o destaque é

⁵As propriedades ACID são um conjunto de quatro pilares fundamentais que garantem a confiabilidade e a integridade dos dados em um SGBD durante o processamento de transações. O acrônimo refere-se a:

- **Atomicidade**: garante que a transação seja tratada como uma unidade indivisível; ou todas as suas operações são confirmadas (`commit`), ou, em caso de falha, todas são revertidas (`rollback`), não permitindo estados parciais.

a Atomicidade:

$$T = \{op_1, op_2, \dots, op_n\} \implies \text{Sucesso total ou Reversão total}$$

Se uma exceção ocorrer, utilizamos `db.session.rollback()` para invalidar a transação e retornar o banco ao último estado consistente.

População de Tabelas de Domínio (Seed)

Iniciaremos populando as entidades que servem de catálogo para o sistema. A persistência de objetos ocorre em duas etapas:

1. adicionar o objeto à sessão;
2. confirmar a transação.

Começamos criando os níveis hierárquicos do sistema.

```
>>> from delivery.models import Level

>>> god = Level(
...     name="God",
...     description="Maior nivel hierarquico"
... )

>>> premium = Level(
...     name="Premium",
```

- **Consistência:** assegura que uma transação leve o banco de dados de um estado válido para outro estado válido, respeitando todas as regras, restrições e gatilhos definidos no esquema.
- **Isolamento:** garante que transações simultâneas não interfiram umas nas outras, fazendo com que o resultado de execuções concorrentes seja o mesmo que se tivessem sido executadas sequencialmente.
- **Durabilidade:** garante que, uma vez que a transação tenha sido confirmada, seus efeitos sejam permanentes e persistam mesmo em caso de falhas de energia ou travamentos do sistema.

```
...     description="Nivel intermediario superior"
... )

>>> basic = Level(
...     name="Basic",
...     description="Nivel intermediario basico"
... )

>>> standard = Level(
...     name="Standard",
...     description="Nivel padrao do sistema"
... )

>>> db.session.add_all([
...     god,
...     premium,
...     basic,
...     standard
... ])

>>> db.session.commit()
```

Agora criaremos alguns papéis da plataforma:

```
>>> from delivery.models import Role

>>> admin_role = Role(name="Administrador")

>>> manager_role = Role(name="Gerente")

>>> operator_role = Role(name="Operador")

>>> client_role = Role(name="Cliente")

>>> db.session.add_all([
...     admin_role,
```



```

...     manager_role,
...     operator_role,
...     client_role
... ])

>>> db.session.commit()

```

Criando Tipos de Negócio (Normalização)

```

>>> from delivery.models import Business
>>> b_types = [
...     BusinessType(name="Pizzaria"),
...     BusinessType(name="Hamburgueria")
... ]
>>> db.session.add_all(b_types)
>>> db.session.commit()

```

Em seguida, criaremos dois usuários:

```

>>> from delivery.models import User

>>> joao = User(
...     name="Joao Silva",
...     email="joao@email.com",
...     cpf="111.111.111-11"
... )
>>> maria = User(
...     name = "Maria Mara",
...     email="maria@email.com",
...     cpf="222.222.222-22")
>>> db.session.add_all([joao, maria])
>>> db.session.commit()

```

Agora criaremos duas empresas distintas:

```
>>> from delivery.models import Business

>>> pizza_type = BusinessType.query.filter_by(name="Pizzaria"↵
↵).first()

>>> burger_type = BusinessType.query.filter_by(name="↵
↵Hamburgueria").first()

>>> pizza_business = Business(
...     owner=joao,
...     corporate_name="Pizzaria Joao LTDA",
...     trade_name="Pizzaria Suprema",
...     cnpj="11.111.111/0001-11",
...     business_type=pizza_type
... )

>>> burger_business = Business(
...     owner=joao,
...     corporate_name="Burger Joao LTDA",
...     trade_name="Burger House",
...     cnpj="22.222.222/0001-22",
...     business_type=burger_type
... )

>>> db.session.add_all([pizza_business, burger_business])
>>> db.session.flush() # Gera os IDs das empresas sem ↵
↵encerrar a transacao

>>> joao_pizza_access = RoleUser(
...     user=joao,
...     role=admin_role,
```

```

...     business=pizza_business,
...     level=premium
... )

>>> joao_burger_access = RoleUser(
...     user=joao,
...     role=admin_role,
...     business=burger_business,
...     level=premium
... )

>>> db.session.add_all([joao_pizza_access, joao_burger_access↵
↵])
>>> db.session.commit()

```

Persistência de Entidades Relacionadas

Agora, testaremos a criação de um grafo de objetos complexo, envolvendo User, Business e Address.

```

>>> # Criando o proprietario
>>> u = User(name="Admin", email="admin@delivery.com", cpf="↵
↵123.456.789-10")
>>> db.session.add(u)
>>> db.session.flush() # Flush gera o ID sem encerrar a ↵
↵transacao
>>>
>>> # Criando a empresa vinculada ao usuario e ao tipo de ↵
↵negocio
>>> pizza_type = BusinessType.query.filter_by(name="Pizzaria"↵
↵).first()

```

```
>>> biz = Business(
...     owner=u,
...     trade_name="Pizza Express",
...     cnpj="001.002.003/0001-01",
...     business_type=pizza_type
... )
>>>
>>> # Testando o Polimorfismo do Address (Endereco da Empresa ↵
    ↵)
>>> c = City(name="Vila Velha", state="ES")
>>> addr = Address(road="Rua A", number=100, business=biz, ↵
    ↵city=c)
>>>
>>> db.session.add_all([biz, c, addr])
>>> db.session.commit()
```

Testando Relacionamentos e Contextos Organizacionais

A principal característica da modelagem atual é que um mesmo usuário pode exercer múltiplos papéis em múltiplas empresas simultaneamente.

Além disso o nível hierárquico pertence ao vínculo, o papel depende do contexto organizacional e o relacionamento possui temporalidade própria. Tudo isso é representado pela entidade RoleUser

Criemos inicialmente João como administrador da pizzaria:

```
>>> from delivery.models import RoleUser

>>> joao_pizza_admin = RoleUser(
...     user=joao,
...     role=admin_role,
```

```

...     business=pizza_business,
...     level=god
... )

>>> db.session.add(joao_pizza_admin)
>>> db.session.commit()

```

Agora João será gerente da hamburgueria:

```

>>> joao_burger_manager = RoleUser(
...     user=joao,
...     role=manager_role,
...     business=burger_business,
...     level=premium
... )

>>> db.session.add(joao_burger_manager)
>>> db.session.commit()

```

Além disso, João também pode atuar como cliente comum da plataforma:

```

>>> joao_client = RoleUser(
...     user=joao,
...     role=client_role,
...     business=None,
...     level=standard
... )

>>> db.session.add(joao_client)
>>> db.session.commit()

```

Observe um aspecto importante:

- o papel de cliente não depende de empresa;
- por isso, `business=None`.

Isso demonstra a diferença entre:

- papéis globais;
- papéis contextualizados por empresa.

Essa modelagem resolve uma limitação importante de sistemas simplificados.

Agora:

- um usuário pode possuir o mesmo papel em empresas diferentes;
- cada vínculo possui nível próprio;
- cada associação possui validade temporal independente.

Por exemplo:

(User, Role, Business)

passa a representar semanticamente uma atuação específica do usuário dentro da plataforma.

Testando Restrições de Unicidade

A modelagem também estabelece restrições semânticas importantes.

Por exemplo, um usuário não deve possuir simultaneamente o mesmo papel na mesma empresa.

Tentemos criar uma duplicação:

```
>>> duplicated = RoleUser(  
...     user=joao,
```

```

...     role=manager_role ,
...     business=burger_business ,
...     level=basic
... )

>>> db.session.add(duplicated)
>>> db.session.commit()

```

Dependendo da restrição definida via:

```
1 UniqueConstraint
```

o banco deverá impedir a duplicação.

Isso garante integridade lógica da associação:

$$(User, Role, Business) \rightarrow \text{tupla unica}$$

Atualizando Dados

Objetos persistidos podem ser modificados diretamente em memória.

Por exemplo:

```

>>> manager_role.name = "Gerente Operacional"

>>> db.session.add(manager_role)
>>> db.session.commit()

```

Também podemos alterar níveis hierárquicos:

```

>>> joao_burger_manager.level = god

>>> db.session.add(joao_burger_manager)

```

```
>>> db.session.commit()
```

Nesse caso, João passa a possuir maior nível hierárquico na hamburgueria.

Encerrando Vínculos

Em vez de remover associações diretamente, muitas vezes é mais interessante preservar histórico.

Para isso utilizamos:

```
1 finished_at
```

Por exemplo:

```
>>> from datetime import datetime

>>> joao_burger_manager.finished_at = datetime.utcnow()

>>> db.session.add(joao_burger_manager)
>>> db.session.commit()
```

Agora o vínculo permanece registrado no banco, mas é interpretado como encerrado.

Essa abordagem preserva rastreabilidade histórica das operações do sistema.

Removendo Dados

A remoção física de objetos ocorre por meio de:

```
1 db.session.delete()
```

Por exemplo:


```
>>> db.session.delete(operator_role)
>>> db.session.commit()
```

Assim como inserções e atualizações, remoções somente são efetivadas após:

```
1 commit()
```

Dependendo das regras de cascata definidas nos relacionamentos, remoções podem impactar entidades associadas.

Consultando Dados

Cada modelo possui automaticamente um objeto de consulta:

```
1 Model.query
```

A consulta mais simples consiste em recuperar todos os registros:

```
>>> Role.query.all()
```

Resultado esperado:

```
[
  <Role Administrador>,
  <Role Gerente Operacional>,
  <Role Operador>,
  <Role Cliente>
]
```

Também podemos recuperar usuários:

```
>>> User.query.all()
```

Ou aplicar filtros específicos:

```
>>> User.query.filter_by(email="joao@email.com").first()
```

Consultas podem ainda explorar relacionamentos:

```
>>> RoleUser.query.filter_by(
...     business=burger_business
... ).all()
```

Ou recuperar todos os vínculos ativos:

```
>>> RoleUser.query.filter_by(
...     finished_at=None
... ).all()
```

O ORM converte automaticamente essas operações em consultas SQL equivalentes.

Navegando entre Relacionamentos

Como os relacionamentos foram definidos com:

```
1 relationship()
```

a navegação entre entidades torna-se natural.

Por exemplo:

```
>>> joao.role_associations
```

retorna todos os vínculos associados ao usuário.

Também podemos acessar diretamente os papéis:

```
>>> joao.roles
```

Ou recuperar os negócios associados:

```
>>> joao.businesses
```

Da mesma forma:

```
>>> pizza_business.role_associations
```

permite recuperar todos os usuários vinculados à pizzaria.

Essa navegação bidirecional constitui uma das principais vantagens do ORM, pois transforma o banco relacional em um grafo navegável de objetos Python.